UMBC Parking Management System

Semester Project – Assignment 6

System focus	DBMS	Core workflow
Campus parking operations	PostgreSQL	Users, permits, reservations, tickets, payments

1. Requirements Document

1.1 Parking users and roles

- Student: Registers vehicles, buys or receives eligible permits, reserves allowed spots, views tickets, and pays citations.
- Faculty/Staff: Registers vehicles, holds staff permits, reserves eligible spots, views tickets, and pays citations.
- Visitor: May register temporary vehicle information, obtain short term parking, reserve visitor spots when enabled, and pay citations.
- Parking Admin: Maintains lots, spots, permit types, pricing, ticket records, and administrative overrides.
- Enforcement Officer: Checks permit/ticket status, issues citations, and records spot/vehicle observations.

1.2 Role-permission matrix

Permission / Action	Student	Faculty/Staff	Visitor	Admin	Officer
Create/update own profile	Y	Y	Y	Y	Y
Register vehicle	Y	Y	Y	Y	N
Hold long-term permit	Y	Y	N	N	N
Hold visitor/day permit	N	N	Y	N	N
Reserve spot	Y	Y	Y*	Y	N
View own reservations	Y	Y	Y	Y	N
View/pay own tickets	Y	Y	Y	Y	N
Issue citation	N	N	N	Y	Y
Manage lots/spots	N	N	N	Y	N
Manage permit types/rules	N	N	N	Y	N
Override reservation/ticket status	N	N	N	Y	N
Run operational reports	N	N	N	Y	Y

** Visitor reservations are limited to visitor enabled spots and configured time windows.*

1.3 System modules to implement

- User and role management
- Vehicle registration
- Permit type management
- Permit issuance and active/expired status tracking
- Lot and spot inventory management
- Reservation management with time windows
- Citation (ticket) issuance and status tracking
- Payment recording for citations
- Administrative reporting views such as active permits, current occupancy, reservation history, and unpaid tickets

2. Business Rules and Constraints

The following rules define required integrity behavior. Type A means the rule is primarily enforced with table constraints such as PK, FK, UNIQUE, or CHECK. Type B means the rule needs a trigger or stored function because it depends on time overlap, cross-row logic, or workflow state changes.

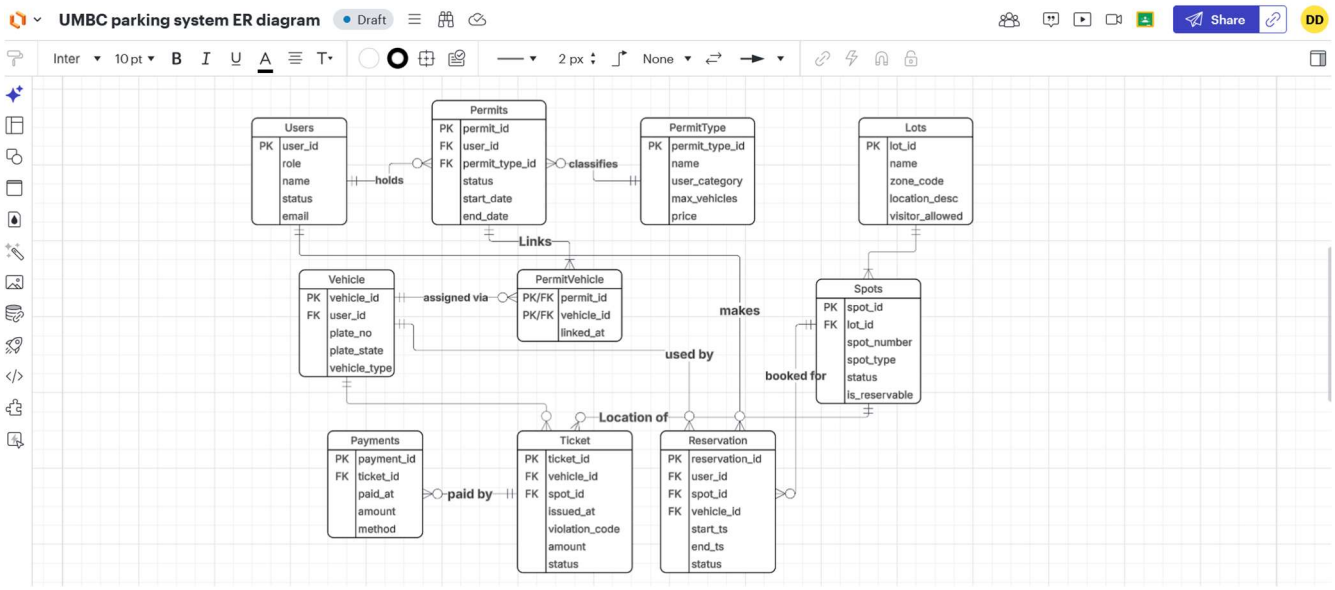
Type	Business rule	Suggested enforcement
A	Each user must have a unique user_id.	PK on Users.user_id
A	Each vehicle plate number + plate_state combination must be unique.	UNIQUE (plate_no, plate_state)
A	Every vehicle must belong to exactly one user.	FK Vehicles.user_id -> Users.user_id NOT NULL
A	Each permit must reference one valid permit type.	FK Permits.permit_type_id
A	Permit end_date must be on or after start_date.	CHECK (end_date >= start_date)
A	Each spot number must be unique within a lot.	UNIQUE (lot_id, spot_number)
A	Reservation end time must be later than reservation start time.	CHECK (end_ts > start_ts)
A	Ticket amount and payment amount must be nonnegative.	CHECK (amount >= 0)
A	A payment must reference a valid ticket.	FK Payments.ticket_id -> Tickets.ticket_id
A	PermitVehicle can contain the same permit_id and vehicle_id pair only once.	Composite PK (permit_id, vehicle_id)
B	A vehicle cannot be linked to a permit if the permit is expired or cancelled.	Trigger/function validates permit status and dates before insert
B	The number of vehicles linked to a permit cannot exceed the max_vehicles defined by the permit type.	Trigger counts linked vehicles before insert into PermitVehicle
B	A user cannot reserve the same spot for an overlapping time window already reserved by another active reservation.	Trigger or exclusion logic checks overlap by spot and time
B	A reservation can be created only for reservable spots and only when the vehicle belongs to the same user making the reservation.	Trigger validates spot flag and ownership
B	A ticket should not be marked paid until cumulative payments are at least the ticket amount.	Trigger/function updates ticket status when payment threshold is reached
B	Only visitor-enabled spots may be used by visitor reservations or visitor permits when such rules apply.	Trigger validates spot and permit/role compatibility

3. Concurrency Scenario

Scenario: Two users try to reserve the same parking spot for overlapping times at nearly the same moment.

Required result: only one reservation succeeds. The first transaction that acquires the lock and passes the overlap check commits. The other transaction must fail and return a clear error such as “spot already reserved for the requested time window.” This prevents double booking and keeps reservation data consistent under concurrent access.

4. Conceptual ER Diagram



5. Design Rationale

PermitType is modeled separately from Permit because permit categories such as student commuter, faculty/staff, or visitor day pass describe reusable policy data rather than one specific permit record. Keeping PermitType separate avoids repeating values such as allowed user category, pricing, and maximum linked vehicles in every permit row. Permit stores the individual instance actually issued to a user, with its own dates and status. The same separation is used for Lots and Spots so that lot level properties and spot level properties can be managed independently.

Reservations are modeled with explicit start and end timestamps because parking availability is time-based rather than only date based. That structure allows the system to detect overlap conflicts and support short reservations, visitor windows, and future occupancy reports. Tickets and Payments are kept separate because one citation may be paid in more than one transaction and payment history should remain auditable. The PermitVehicle bridge entity resolves the many to many relationship between permits and vehicles while still allowing enforcement rules such as a maximum number of vehicles per permit type.

6. Logical Design

The logical schema below follows the entities and relationships from Assignment 2 and is designed so it can be implemented directly in PostgreSQL using primary keys, foreign keys, unique constraints, check constraints, and triggers where needed.

6.1 Relational schema list

- Users(PK user_id, role, name, email, status)
- PermitType(PK permit_type_id, name, user_category, max_vehicles, price)
- Lots(PK lot_id, name, zone_code, location_desc, visitor_allowed)
- Vehicles(PK vehicle_id, user_id, plate_no, plate_state, vehicle_type, FK -> Users(user_id))

- Permits(PK permit_id, user_id, permit_type_id, status, start_date, end_date, FK -> Users(user_id), FK -> PermitType(permit_type_id))
- Spots(PK spot_id, lot_id, spot_number, spot_type, status, is_reservable, FK -> Lots(lot_id))
- PermitVehicle(PK permit_id, PK vehicle_id, linked_at, FK -> Permits(permit_id), FK -> Vehicles(vehicle_id))
- Reservations(PK reservation_id, user_id, spot_id, vehicle_id, start_ts, end_ts, status, FK -> Users(user_id), FK -> Spots(spot_id), FK -> Vehicles(vehicle_id))
- Tickets(PK ticket_id, vehicle_id, spot_id, issued_at, violation_code, amount, status, FK -> Vehicles(vehicle_id), FK -> Spots(spot_id))
- Payments(PK payment_id, ticket_id, paid_at, amount, method, FK -> Tickets(ticket_id))

6.2 Candidate keys and composite keys

- Users: candidate key = user_id.
- PermitType: candidate key = permit_type_id.
- Lots: candidate key = lot_id.
- Vehicles: candidate keys = vehicle_id and (plate_no, plate_state). The surrogate key vehicle_id is used as
- the primary key, while the plate and state pair is kept unique.
- Permits: candidate key = permit_id.
- Spots: candidate keys = spot_id and (lot_id, spot_number). spot_id is the primary key, while the lot and
- spot number pair stays unique inside each lot.
- PermitVehicle: composite primary key = (permit_id, vehicle_id). This is a bridge table for the many to
- many relationship between permits and vehicles, so one column alone is not enough to uniquely identify a
- row.
- Reservations: candidate key = reservation_id.
- Tickets: candidate key = ticket_id.
- Payments: candidate key = payment_id.

6.3 Normalization Discussion

Table Pair 1: PermitType and Permits
If permit type details were stored directly in the Permits table, attributes such as name, user_category, max_vehicles, and price would be repeated in many rows. The functional dependencies would look like this:

- permit_id -> user_id, permit_type_id, status, start_date, end_date
- permit_type_id -> name, user_category, max_vehicles, price

This creates a problem because name, user_category, max_vehicles, and price are not properties of one specific permit record. They belong to the permit type. That means some non-key attributes depend on another non-key attribute, which creates a transitive dependency.

Anomaly:
For example, if the price of a permit type such as Student Commuter changes, then every permit row with that type would need to be updated. If some rows are updated and others are missed, the same permit type could end up with different prices in different rows. Inserting a new permit type would also be difficult unless at least one permit of that type already exists. Similarly, if the last permit of a certain type is deleted, the information about that permit type could be lost.

This issue is avoided by separating PermitType from Permits. PermitType stores the reusable permit policy data once, while Permits stores only the individual permit issued to a user and references the type through permit_type_id.

Table Pair 2: Lots and Spots
If lot-level attributes such as name, zone_code, location_desc, and visitor_allowed were repeated inside every row of Spots, the functional dependencies would look like this:

- spot_id -> lot_id, spot_number, spot_type, status, is_reservable
- lot_id -> name, zone_code, location_desc, visitor_allowed

This causes the same kind of issue as in Table Pair 1 because lot-level attributes would be repeated for every spot in the same lot instead of being stored once.

Anomaly:
For example, if a lot changes from visitor enabled to not visitor enabled, every spot row in that lot would have to be updated. If even one row is missed, the data becomes inconsistent. By storing Lots and Spots separately, lot facts are kept once at the lot level and spot facts are kept once at the spot level. This reduces redundancy and keeps the design normalized.

6.4 Naming conventions

• Table names use business entities with consistent capitalization, for example Users, Vehicles, Permits, and Tickets.

• Primary key columns end in _id, for example user_id, vehicle_id, permit_id, and ticket_id.

• Foreign key columns reuse the referenced primary key name.

• Columns use lowercase with underscores, for example plate_state, start_date, and violation_code.

• Date-only attributes use the suffix _date. Timestamp attributes use _ts or _at depending on the meaning.

• Boolean columns use readable names such as is_reservable and visitor_allowed.

• Status columns are named status for consistency across permits, reservations, tickets, and users.

6.5 Sanity Run

Script execution order :

1. createDDL.sql
2. loadAll.sql
3. querey to check data was loaded.
4. Check enforcement of uniqueness constraints.
5. dropDDL.sql

7. Indexing/Optimization

I used EXPLAIN (ANALYZE, BUFFERS) on 3 expensive queries before and after creating indexes, and the outputs of these queries are included in the CSV files I submitted. For the unpaid ticket balance query, the

estimated cost dropped from 57.10..57.11 to 11.39..11.40 and execution time improved from 0.452 ms to 0.188 ms. For the reservation conflict query, the estimated cost dropped from 22.09..22.09 to 1.19..1.19 and execution time improved from 0.123 ms to 0.039 ms. For the latest sensor event query, the estimated cost dropped from 48.40..49.15 to 6.01..6.01, and the plan changed to use an index scan, but the execution time changed only slightly from 0.061 ms to 0.074 ms. Since the sample data is small PostgreSQL still used sequential scans on some tables, which is normal. Overall the indexes improved the plans and helped 2 of the 3 queries more clearly.

8. Final User Guide and Transaction Demo

8.1 Final package

The final package includes docker-compose.yml, README.md, dropDDL.sql, createDDL.sql, loadAll.sql, queryAll.sql, indexAll.sql, transaction.sql, and the final report PDF. The database can be rebuilt by following the README.

8.2 Concurrency demo

The demo shows double booking problem. In the first part double booking problem is demonstrated by making a reservation for the same spot using two separate overlapping sessions. Both sessions are able to insert and commit, since no lock is used causing the double booking.

The second part shows how double booking can be prevented by getting one of the sessions to lock the spot row using SELECT ... FOR UPDATE. After the first session is done and the row is unlocked the second session is able to see the conflict and fail to insert instead of a double booking happening.

8.3 System Walkthrough

Permit issuance is tested with the issue_permit function, which checks the user, permit type, dates, and eligibility before inserting a permit.

Reservation is tested by inserting a reservation with a user, vehicle, spot, start time, and end time.

Sensor occupancy is tested by inserting into `sensor_events`. The trigger updates the related spot status.

Ticket creation is tested by running `CALL auto_ticketing(...)`, which creates a ticket for a vehicle without a valid permit.

Payment is tested by inserting a row into `payments` and updating the related ticket when the required amount is paid.

8.4 Screenshots included

The report includes screenshots for setup, table creation, sample data, constraint testing, and script execution from the earlier sections. For Assignment 6, the concurrency demo is shown through transaction.sql and the two-session pgAdmin test in figure 6, 7 and 8.

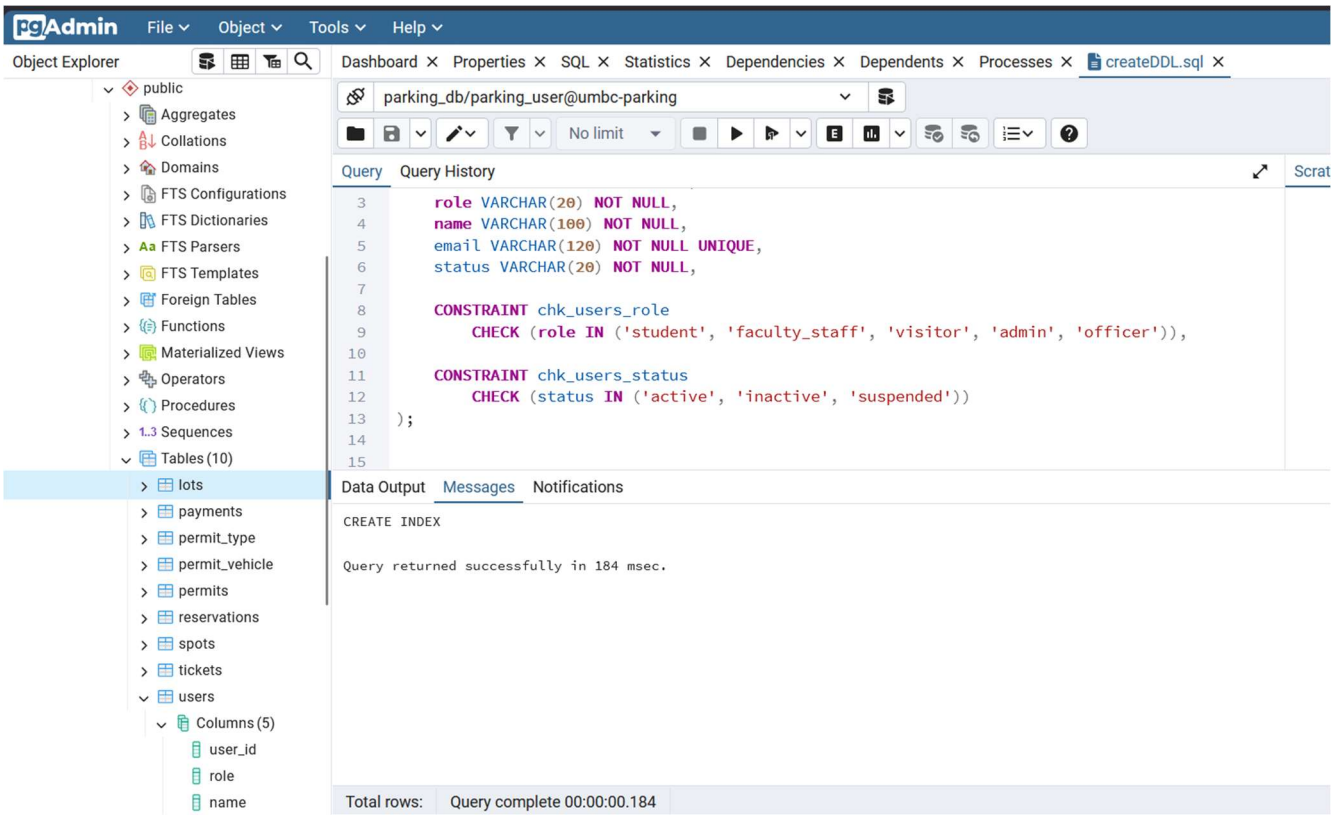


Figure 1: Successful execution of createDDL.sql

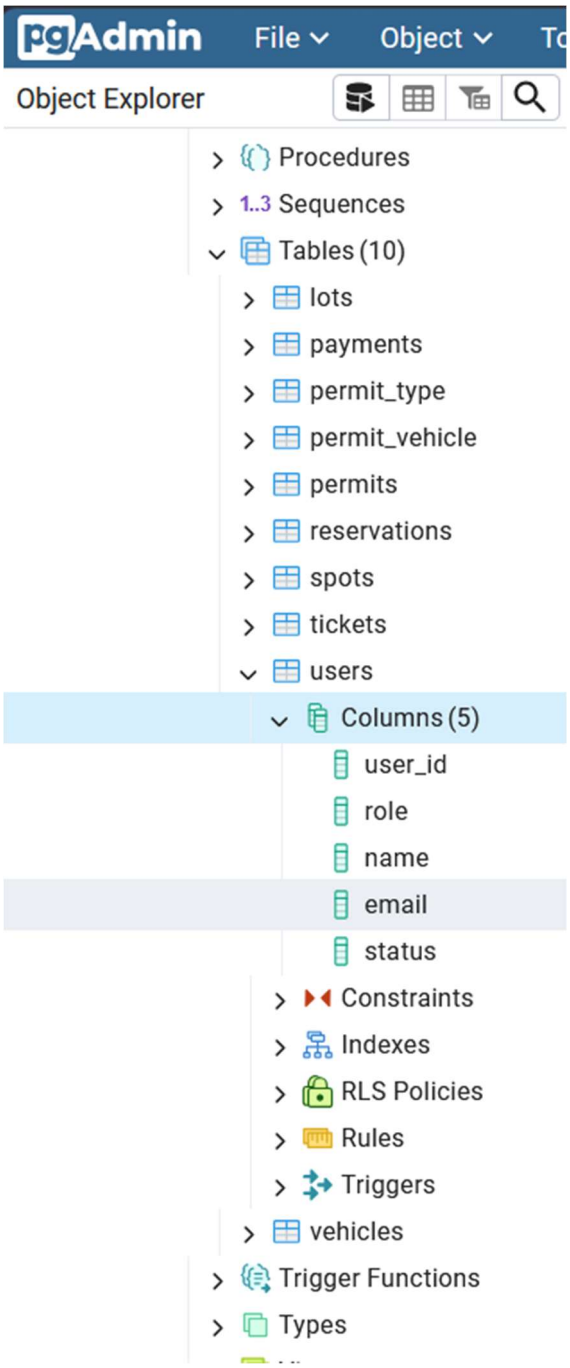


Figure 2**:** All required tables are present in the database.

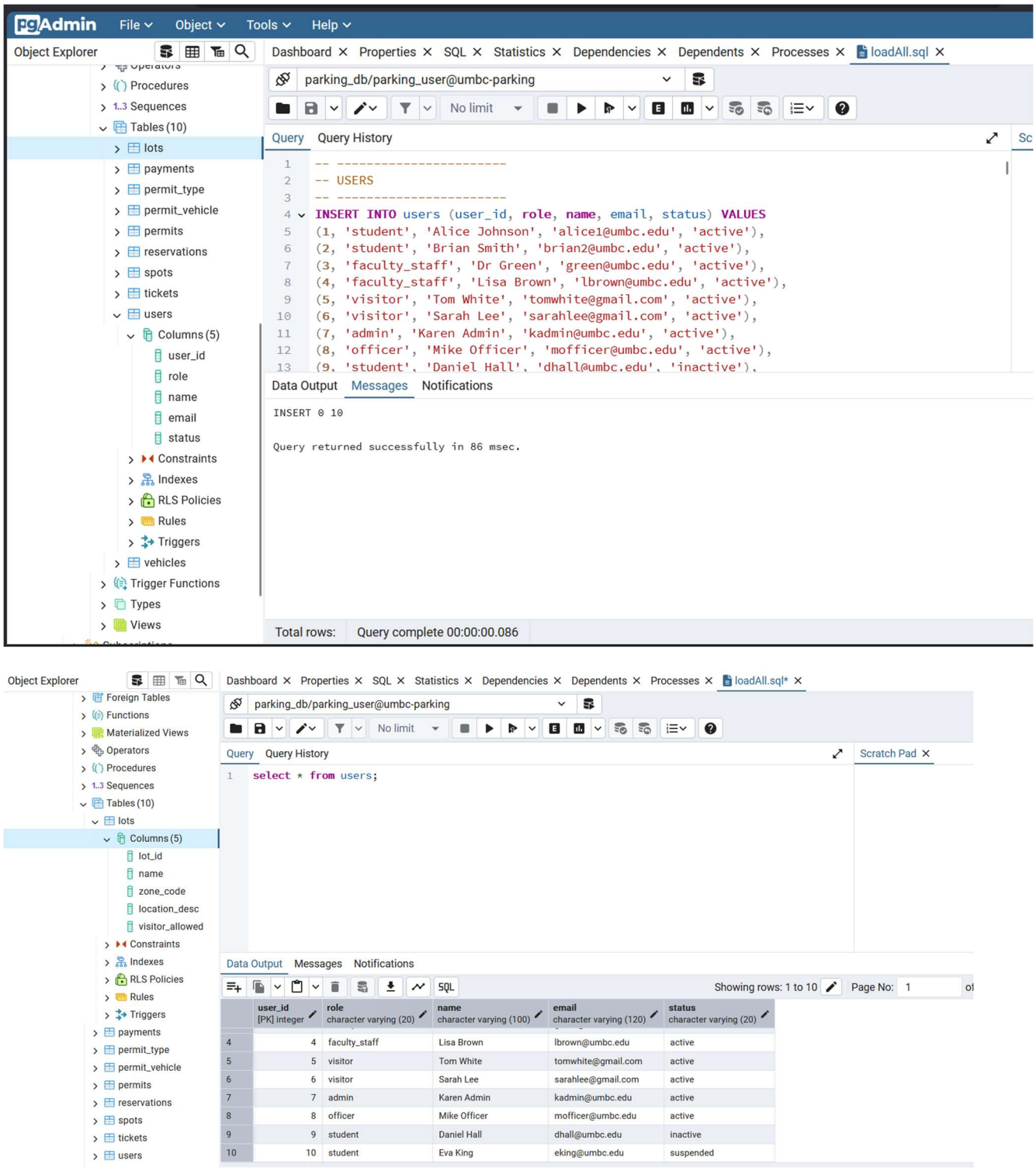


Figure 3**:** Sample data was inserted correctly into the database.

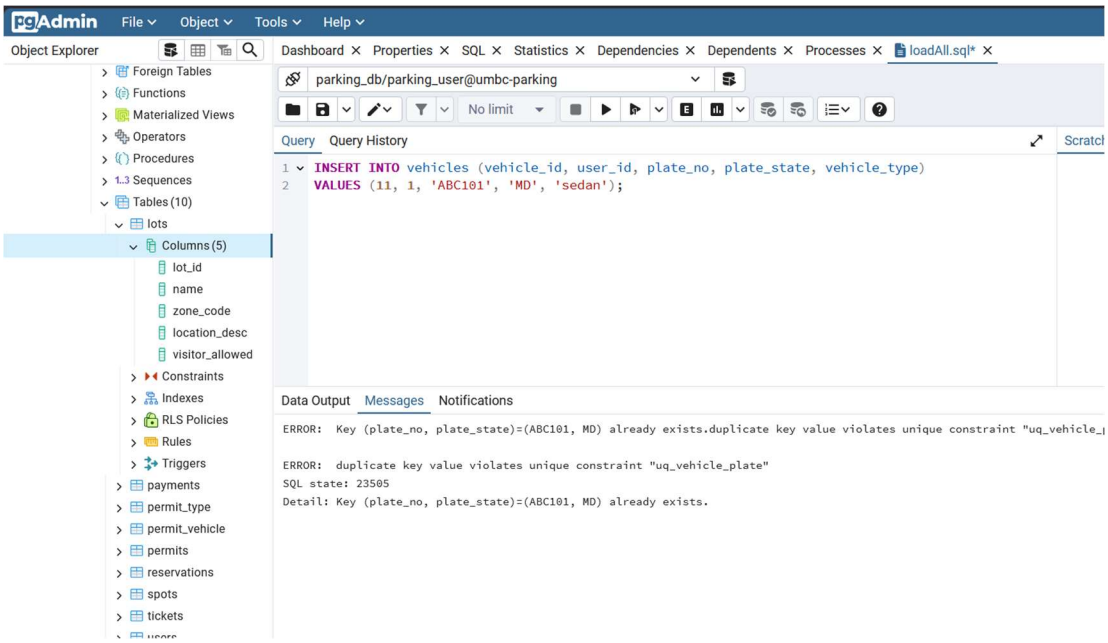


Figure 4: The database correctly enforces uniqueness constraints by rejecting duplicate entries.

The UNIQUE constraint on (plate_no, plate_state) was validated by attempting a duplicate insert, which correctly failed.

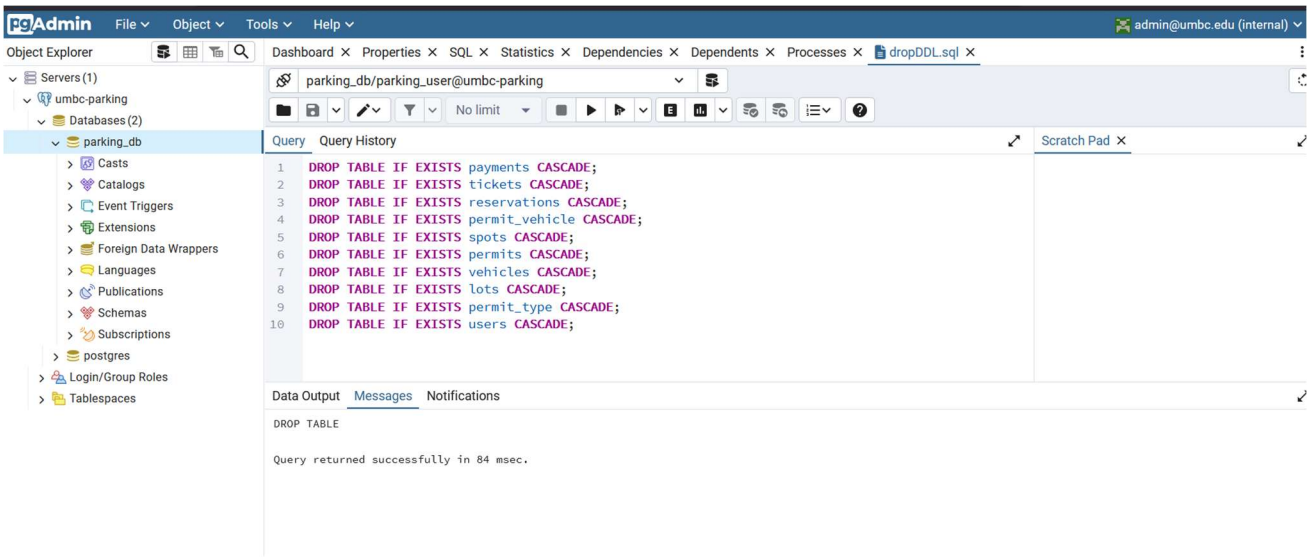


Figure 5: Successful execution of dropDDL.sql

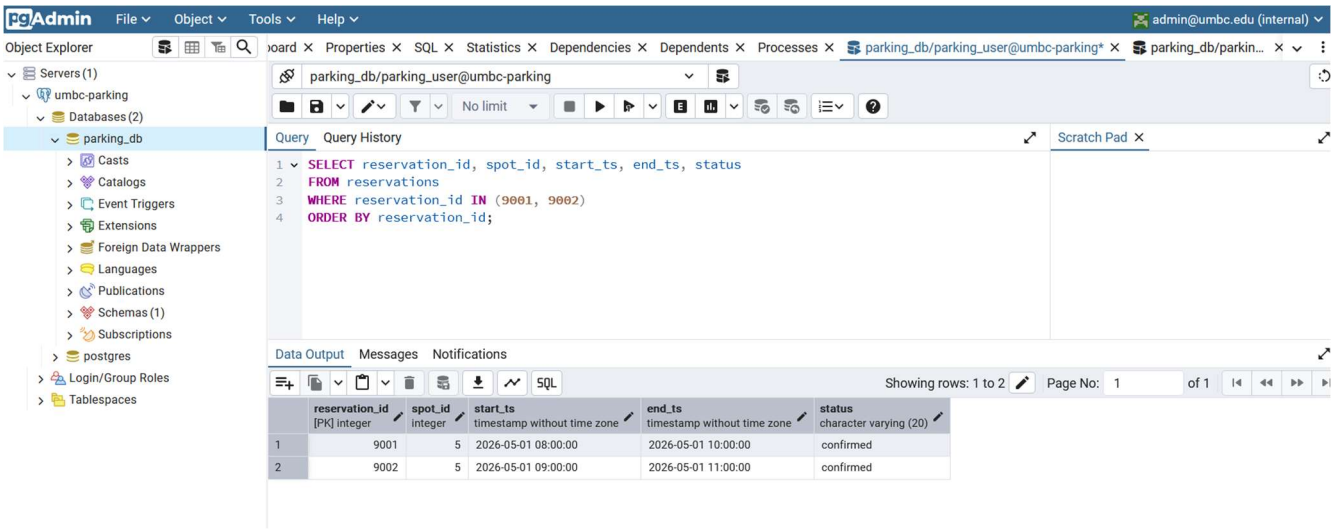


Figure 6: Double-booking issue

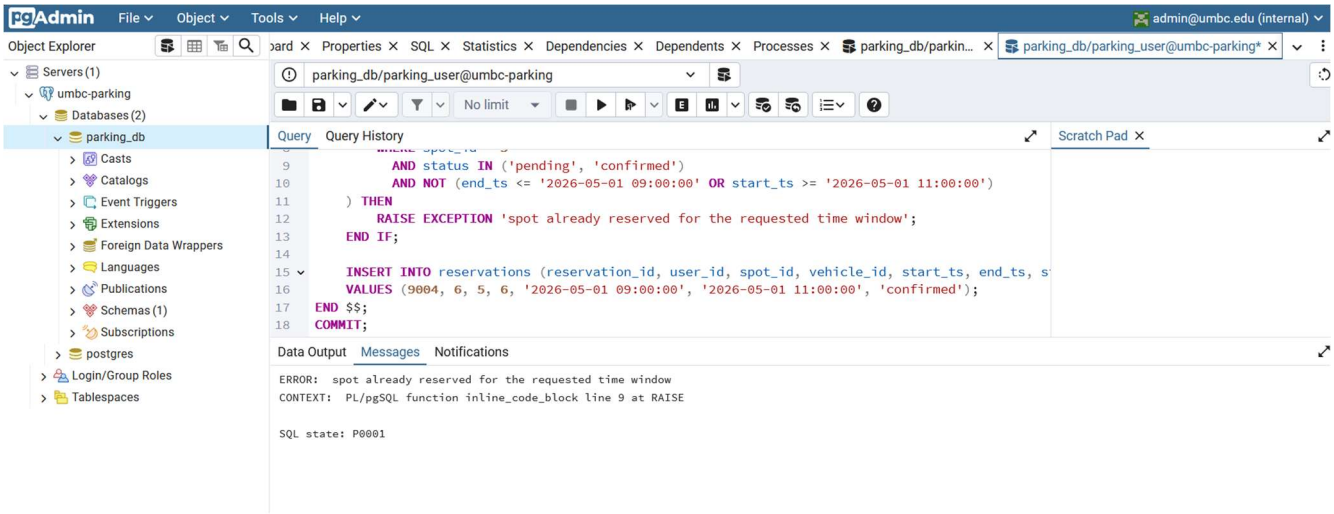


Figure 7: Double-booking prevented

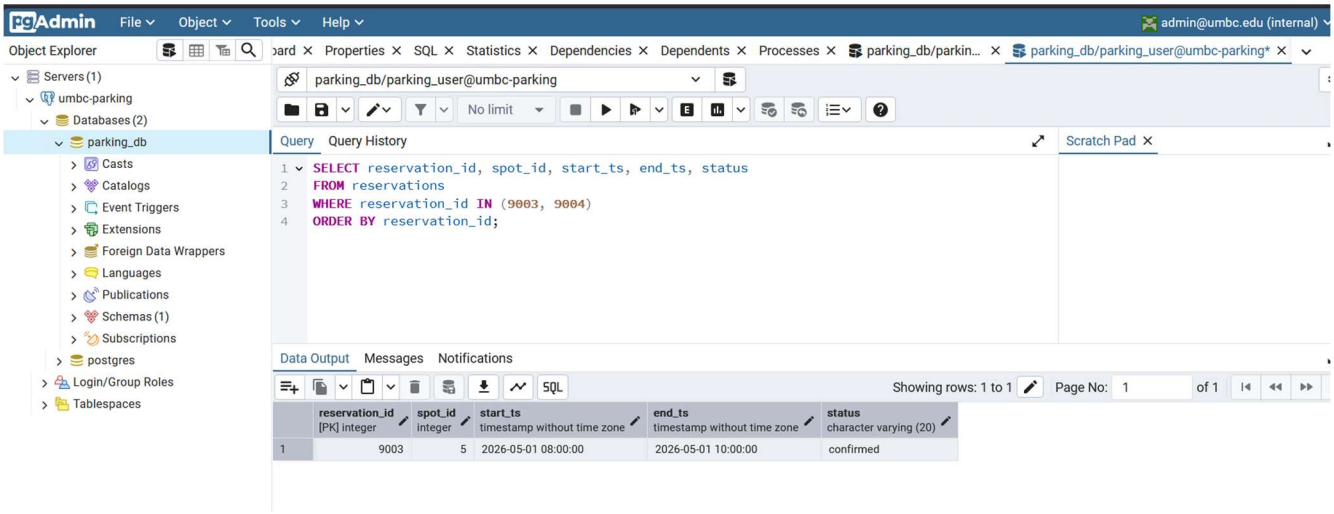


Figure 8: Only first reservation shows